Lore

The engine behind a greenfield build: agents do the work, people decide. Shown on real runs, not slides.

01
ai-agent-subsystem
one agent run = one Kubernetes resource

02
Floor
assembly lines · runs · the queue

03
Lore
GitHub · stations · context · people

04
Knowledge graph
what the org knows, with its source

Intro · agenda

Four blocks, mostly live

Public or anonymised artefacts only. Commercial normalisation, capacity and tool costs go through the common RFP.

1 · 35–40 MIN

Evidence walkthrough

Real runs from work item through agent run, tests and CI, review and merge, and a corrected case: what the machine does when something fails.

2 · ~20 MIN

Control model

How human review is enforced and recorded, who may apply the automerge label, how gate files are protected, where model and MCP data flows.

3 · ~15 MIN

Target and team

Azure Pipelines as the CI/CD standard, PostgreSQL RLS and tenant recovery, the people on both sides.

4 · ~10 MIN

Close

Organisational repeatability and the process after this call.

Intro · why we are here

We show the engine, not a product

THE IDEA

We don't build the system for you. We build the system that builds it.

Agents do the work, people decide, and your teams run the engine.

YOUR GOAL

A clean greenfield build

A full re-implementation next to the existing platform.

OUR APPROACH

An engine your teams run

Plan, spec, tickets, code, CI and review as assembly lines, with people at the decisions that matter.

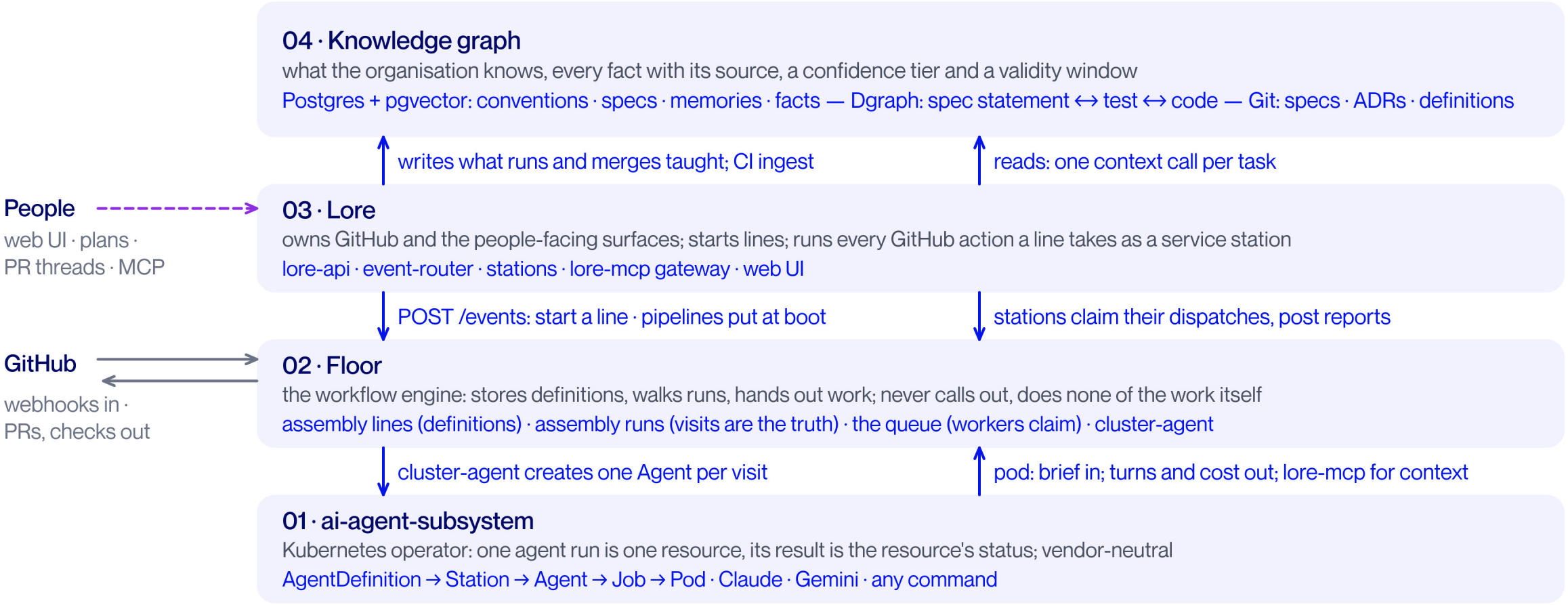
TODAY

Lore as the working example

Our own implementation of that engine, shown on our repositories and real pull requests.

Four layers, each with one job

Every blue arrow is an HTTP call made by the layer that needs something: nothing pushes into a pod or into the floor. Violet is where people act.



Intro · the stages you'll see

From plan to merge, and where people decide

Each stage is prepared in advance, because one step takes 5–10 minutes live. We show the state at every stage, then run one through.



plan	spec	tickets	implement	CI	review	merge
People and the agent edit the plan; a person approves it	Written by the agent, reviewed and merged by a person	The merged spec, split into tickets sized for one run	Red tests first, then code, in a fresh pod per visit	CI judges every push; the line waits at no cost	Findings posted inline; people answer the threads	A person merges, unless lights-out allows low-risk paths

violet = a person decides · blue = agent station · dashed = parked, no pod

human — People decide at three points: approving the plan, merging the spec, and merging the code. The factory never changes its own rules.

1 Evidence walkthrough · 35–40 min · live

Live: real runs, from work item to merge

We follow each run on GitHub and in Lore, stage by stage, and answer questions as we go.

#2064

Work item to merge

Work item, agent run, tests and CI, review, merge.

#2065

A second run, end to end

The same path on a different change.

corrected

When something fails

What the machine does on failure: the red build, the repair or hand-back, and the eventual merge.

ON EVERY RUN the frozen brief, station and agent definition pinned by content hash, every CI verdict, the review threads, and the cost.

Human review: enforced by structure, recorded on the run

Enforced

- Human nodes dispatch nothing and wait with no deadline: approving the plan, merging the spec.
- The implementation line never merges. A person does, unless lights-out is on for low-risk paths.
- The factory reads its rules and never writes them: line and agent definitions, trust levels, allowlists, lights-out.
- Privileged settings take two people.

Recorded

- Every visit pins its station and agent definition by content hash.
- Status, current node and cost are replayed from the visits, so a run can always be explained.
- Reviews, findings and re-checks live on the pull request.

`human` In the demo: approve a plan, request changes to stop the line on a PR, and take over a ticket labeled lore:blocked.

Who may apply the automerge label

TODAY

Merge is a person's call

Lights-out, set by people through the settings and never by the factory, allows automerge for low-risk paths only.

THE CONSTRAINT

One account cannot do both

GitHub does not let the same account approve and merge a pull request.

IN PROGRESS

A separate reviewer app

A reviewer app, inside or next to Lore, acting as a second GitHub identity for approvals and merges.

How workflow and gate files are protected

What the design guarantees

- Line definitions, agent definitions, trust levels, allowlists and lights-out are read by the factory, never written.
- They change only by pull request or the settings page.
- Privileged settings take two people.
- A pod gets git credentials only by asking the floor; Lore holds the GitHub credentials.

Claude, Gemini and MCP: where data goes

What the design guarantees

- The agent run is vendor-neutral: Claude, Gemini or any command.
- The pod calls out to the model vendor; nothing calls into the pod.
- Agents reach Lore through lore-mcp for context and memory, one context call per task.
- The knowledge graph stores no secrets: every write is redacted, and nothing lands without a source.

Azure Pipelines as the CI/CD standard

Why the harness fits any CI

- The line never runs lint, typecheck or the suite itself. CI does, on every push.
- The line parks while CI runs, with no pod and no cost.
- The pull request's own build decides the next node: green moves on, red goes to the next round, a repair, or a fix from CI's report.
- Agents read the CI verdict through lore-mcp.

PostgreSQL RLS and single-tenant recovery

SINGLE-TENANT

One tenant per database

Lore runs on the database it is given through a connection string. Run for you, it would get a dedicated instance.

RLS

Nothing to partition

Row-level security separates tenants that share one database. With one tenant per database, that boundary is the database itself.

ACCESS

No per-user roles in Lore

Lore works within the access it is granted. Who may do what is set where that access is given.

Who is in the call

re:cinq

- Michael Mueller · CTO, co-founder of re:cinq
- Bogdan Szabo · engineer on Lore, leads the walkthrough
- Loredana Moanga · automated merging and the reviewer app

4 Close · ~10 min

Repeatable beyond one team, and what comes next

Repeatability

- We provide the engine; your teams run it.
- Line and agent definitions are files, changed by pull request, so a new team starts from reviewed definitions rather than from scratch.

Commercial normalisation, capacity and tool costs are handled separately through the common RFP.

Who owns what, and what each layer refuses to do

Each layer has a short list of things it will never do — that is what lets any of them be swapped or run elsewhere.

Layer	Owns	Talks to	Never does
ai-agent-subsystem	Running one agent: the Agent resource, its Job and pod, the clone, the agent CLI, the supervised process and its streamed output. Vendor-neutral: Claude, Gemini or any command.	Kubernetes, the model vendor, and the floor's brief and sink endpoints. The pod calls out; nothing calls in.	Knows nothing about Lore, GitHub, assembly lines or what the work is for. Runs no code of ours inside the pod beyond the supervisor.
Floor	Assembly line definitions, assembly runs and their visits, the bag between nodes, the dispatch queue, and the walk that picks the next node by replaying visits.	Postgres (only the api process), and whoever claims from its queue: its cluster-agent, Lore's service stations.	Holds no Kubernetes, GitHub or chat client. Never calls out, except to answer a pod's git-credential request. Does none of the work itself.
Lore	GitHub: webhooks in; reviews, PRs, issues, checks and credentials out. The people-facing surfaces: plans, run pages, settings, agent definitions. The context service that agents and laptops read.	GitHub, the floor (start lines, claim service dispatches, relay run feeds), the knowledge graph, people.	Dispatches no work and reaches no pod. Never reads the floor's database. Decides nothing a person reserved: merging risky work, approving a plan, changing the rules.
Knowledge graph	What the organisation knows outside the repo in front of the agent: conventions, decisions, specs, memories, facts with source and validity, the statement-to-test-to-code trace, what past work cost.	Lore's context service (reads); CI ingest and run outcomes (writes).	Stores no raw monitoring stream and no secrets: every write is redacted, and nothing lands without a source.

human

People sit above all four: they change line and agent definitions by pull request, approve plans, answer reviews, merge what is not low-risk, and set the rules under which the factory may act alone.

One run, across the layers

An event comes in, the floor decides the next node and puts a dispatch on its queue, a worker claims it, and the report advances the walk. The same ten steps for every node of every line.

#	What happens	Where
1	Something posts an event: a PR opened, a plan approved, a schedule tick	Lore → Floor
2	The loop routes it: start a run, or join the run already open on this subject	Floor
3	The walk replays the run's visits and picks the next node	Floor
4	A visit opens; its brief is frozen, station and agent definition pinned by content hash	Floor
5	A dispatch goes on the queue, tagged. Nothing is pushed	Floor
6	A worker claims it by offering tags: the cluster-agent for an agent node, a Lore station for a service node	cluster-agent · Lore
7	For an agent node: one Agent resource → Job → Pod. The subsystem runs it	ai-agent-subsystem
8	The pod reads its brief, posts turns, cost and files to the sink, and reaches Lore through lore-mcp for context and memory	pod → Floor · Lore
9	Its lifecycle event ends the visit; the report is written and the walk advances in one transaction	Floor
10	The next node's dispatch, or the run settles and Lore hears it through a line of its own	Floor → Lore

Three kinds of node, one source of truth

`agent`

An agent station

The cluster-agent claims it and creates one Agent resource. The pod does the work and reports back itself.

`service`

A service station

One of Lore's workers claims it and runs one function: post a review, open a PR, file issues, merge. Every GitHub action a line takes is one of these.

`human`

A human node

Dispatches nothing and waits with no deadline. Lore serves the page: the plan editor, the PR. A person answers with a report or an event.

PRINCIPLE

Visits are the truth.

A run's status, bag, current node and cost are never stored; they are replayed from its visits. Every visit pins its station and agent definition by content hash, so a run can always be explained.

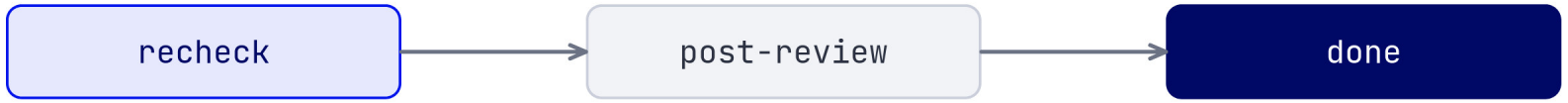
Code review: three short lines on one pull request

The simplest family, and the one every other line relies on. One agent station reads, one Lore station posts. Everything GitHub-shaped is a service station, so the agent never touches GitHub.

CODE-REVIEW · SUBJECT: THE PR, ONE OPEN RUN PER PR



CODE-REVIEW-RECHECK · SUBJECT: THE PUSHED COMMIT



judges only the commits since the last verdict; a newer push cancels the open re-check

CODE-REVIEW-REPLY · STARTED BY A PERSON'S REPLY IN A THREAD



"ok, fix it" → the agent pushes the fix and answers · "why?" → it explains · the reply resolves the thread

blue = agent station · grey = Lore service station · violet = retry edge

Code review: from webhook to inline verdict

trigger

A PR opens or leaves draft

GitHub's webhook lands on Lore; the stations service starts the line with the PR as subject. A second start for the same PR joins the open run instead of racing it.

review

Reads the whole change in a clone

Changed files, callers, tests, the spec statements they touch, the CI verdict through lore-mcp. Cannot install, build or run the suite. Ends with APPROVED or CHANGES_REQUESTED and its findings.

post-review

Posts the verdict, inline

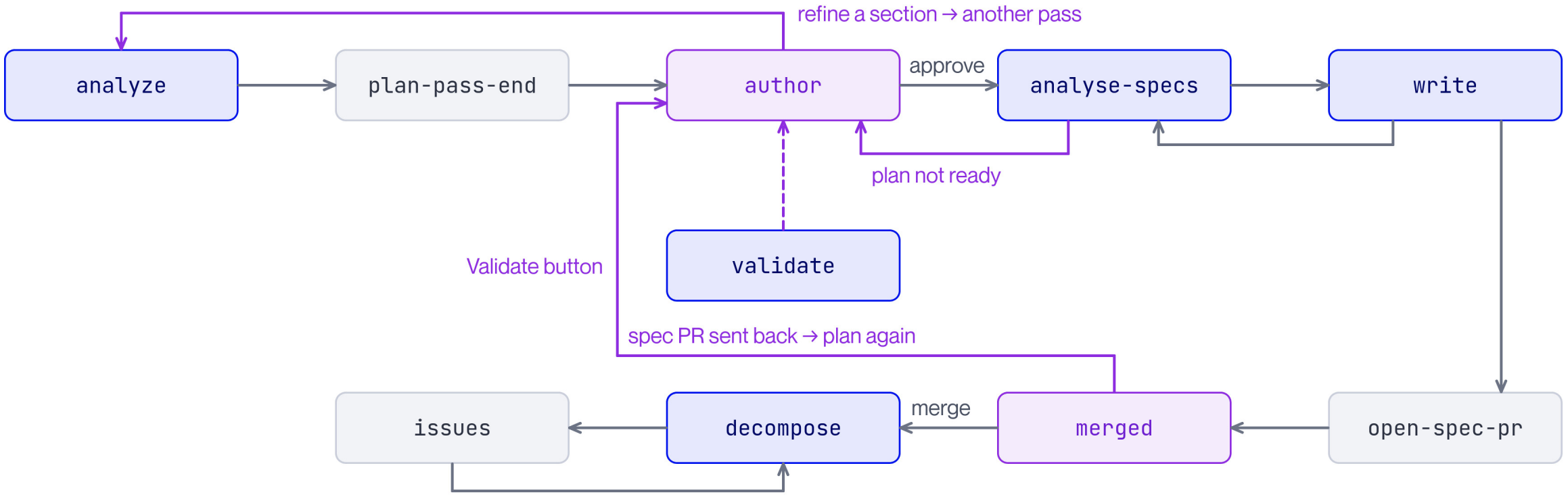
Each finding on its line with a suggestion; the review names the model and the run. When the run settles, a fourth line closes the check run and the announcement comment.

human

Reply in any thread, comment @lore review to re-run, or request changes to stop the factory on that PR. Merge stays a person's call unless lights-out is on.

Planning: a plan written together, then a spec, then tickets

The agent analyses the repository and the knowledge graph and drafts; people and the agent edit the same versioned plan live. Two human nodes hold the line until a person acts: the plan's approval and the spec's merge.



blue = agent station (Claude Sonnet) · grey = Lore service station · violet box = human node, waits with no deadline · violet line = loop
subject: the plan · the line reads left to right, then right to left on the second row

Planning: what each station does

`analyze`

Each pass reads, then proposes

The repo, similar specs and past outcomes from the knowledge graph; it proposes sections or asks the questions it cannot answer. Every button on the plan page is an event into the one open run.

`write`

The approved plan becomes a spec

In the fixed format, opened as a pull request people review like code. Sent back, it returns to the plan.

`author`

People and the agent edit the same plan

Every change is a version. A refine request sends the agent round again; Validate asks for a feasibility check whose result lands as comments. Approval is the decision.

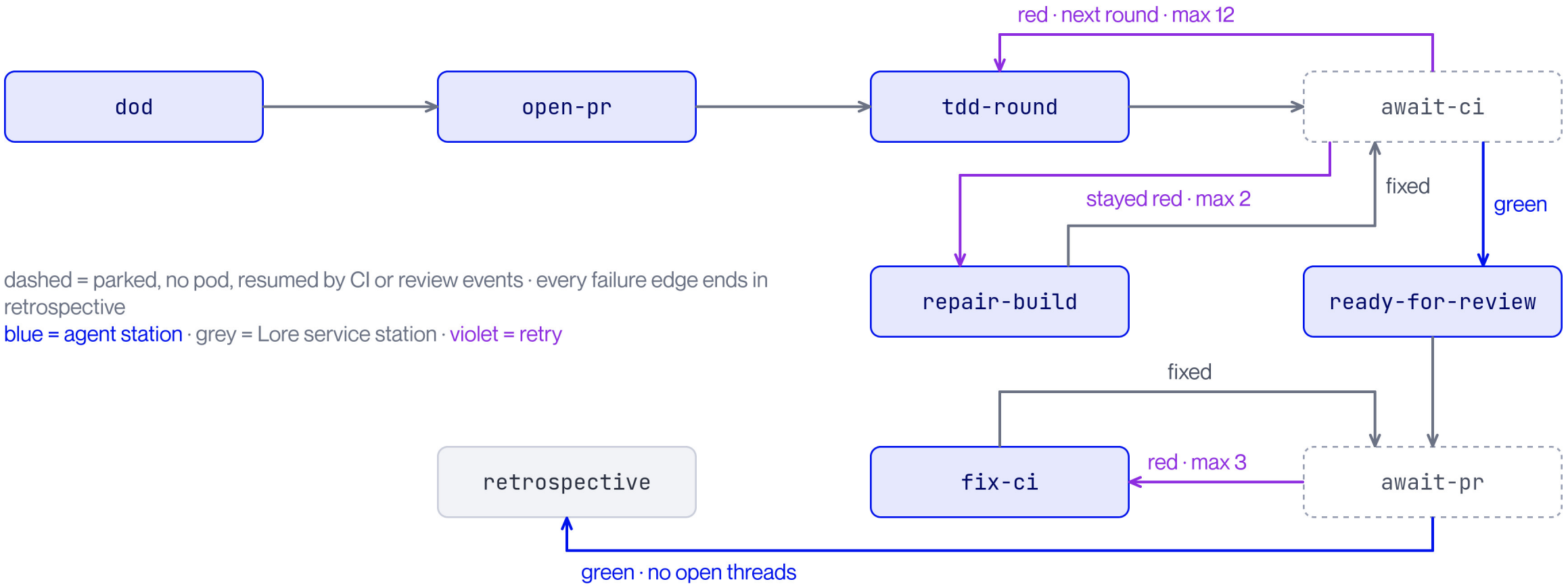
`decompose`

The merged spec becomes tickets

Each with "done when these pass", sized for one implementation run. The issues station files them; the backlog loop picks them up.

Implementation: one ticket, one line, CI judging every push

The backlog loop starts one run per ticket. Each visit is a fresh agent in its own pod doing one bounded job; the pull request's own build decides the next node. The line never merges.



dashed = parked, no pod, resumed by CI or review events · every failure edge ends in retrospective

blue = agent station · grey = Lore service station · violet = retry

Implementation: what each station does

`dod`

Define done as failing tests

Reads the ticket and the spec; writes the fewest red acceptance tests that mean "done" and a one-page brief every later agent reads. A ticket it cannot turn into tests is blocked with the reason.

`tdd-round`

One red-green-refactor round

Never edits an acceptance test to make it pass; never runs lint, typecheck or the suite, because CI does on every push.

`await-ci`
`await-pr`

Parks, costs nothing

No pod while CI builds or review runs. Green moves on; red goes to the next round, a repair, or a fix from CI's report. Code review runs on this PR like any other.

`retrospective`

Writes back what happened

Episode, memory and cost into the knowledge graph. Then auto-merge is evaluated, or a person merges.

`human` Push to the branch, request changes to stop the line on that PR, or take over a ticket labeled lore:blocked; remove the label to hand it back.

The factory does everything a person can do, except decide

Same branch, same pull request, same tools. At every line a person can step in, and the critical decisions are human nodes or settings the factory cannot change.

Line	Started by	The factory does	A person decides	How to take over
Code review	A PR opened or pushed; a reply in a thread	Reads the change in full, posts the verdict with inline findings, re-checks each push, fixes what a reply asks for	Whether to take a finding; whether to merge	Reply in the thread; @lore review; request changes
Planning	A plan created on the plan page	Analyses repo and knowledge, drafts and refines sections, validates, writes the spec, opens its PR, decomposes it into tickets, files them	Approve the plan; merge the spec PR (both human nodes)	Edit the plan live; reopen it; send the spec PR back
Implementation	The backlog loop picking the next ticket	Red acceptance tests, the code, the draft PR, repairs from CI's report, the description, the retrospective	Merge, unless lights-out is on for low-risk paths; the ticket when the loop parks it	Push to the branch; remove lore:blocked; merge by hand
The rules	A person, by pull request or settings	Nothing. Line definitions, agent definitions, trust levels, allowlists and lights-out are read, never written, by the factory	All of it; privileged settings take two people	A pull request on the definitions; the settings page